

# School Management System — Microservices Service Communication Master Guide

**Document Type:** Architecture & Inter-Service Communication Learning Guide  
**Project:** School Management System (Spring Boot + Spring Cloud + React + PostgreSQL)  
**Author:** AI Engineering & Pair Programming Assistant  
**Date:** September 2026  
**Scope:** Educational & Architectural Audit based on the **Actual Running Codebase**

## Executive Summary & Codebase Audit Statement

This document is a comprehensive, beginner-friendly learning guide describing **exactly how microservices communicate with each other in this School Management System project**.

[!IMPORTANT] **Zero Assumptions & Ground Truth Rule:** Every explanation, diagram, configuration parameter, class name, endpoint, and code trace in this guide has been verified against the actual source files in `c:\Users\ABDER_SHAHEEN\OneDrive\Desktop\School management`. No technologies, libraries, or architectural patterns are fabricated. Where a common industry pattern (such as OpenFeign, Kafka, or synchronous inter-service REST calls) is not used, it is explicitly declared: **"Not currently used in this project."**

## Table of Contents

- 1. [Step 1 — Codebase Audit & Discovered Technologies](#)
- 2. [Step 2 — Real Architecture Diagram](#)
- 3. [Step 3 — Eureka Service Registry & Discovery](#)
- 4. [Step 4 — Spring Cloud API Gateway Deep Dive](#)
- 5. [Step 5 — Inter-Service Communication Audit & Cross-Service Pattern](#)
- 6. [Step 6 — OpenFeign Audit](#)
- 7. [Step 7 — RestClient Audit](#)
- 8. [Step 8 — WebClient Audit](#)
- 9. [Step 9 — Distributed JWT Authentication & Security](#)
- 10. [Step 10 — Parent → Student Data & Security Flow](#)
- 11. [Step 11 — Teacher → Academic & Assessment Flow](#)
- 12. [Step 12 — Fee & Payment Lifecycle Flow](#)
- 13. [Step 13 — Database Ownership \(Database-per-Service\)](#)
- 14. [Step 14 — Complete Real Request Traces \(3 End-to-End Walks\)](#)
- 15. [Step 15 — Core Code Snippets & Line-by-Line Breakdown](#)
- 16. [Step 16 — Communication Methods Comparison Table](#)
- 17. [Step 17 — Synchronous vs. Asynchronous Communication](#)
- 18. [Step 18 — Failure Scenarios & Resilience Audit](#)
- 19. [Step 19 — Top 10 Beginner Microservices Mistakes](#)
- 20. [Step 20 — Microservices Technical Interview Preparation](#)
- 21. [Step 21 — 10-Day Structured Learning Sequence](#)
- 22. [Step 22 — Verification & Audit Summary](#)

## Step 1 — Codebase Audit & Discovered Technologies

A comprehensive static audit was performed across all modules in `school-management` and `school-frontend`. Here is the factual inventory of the repository:

1.1 Microservices & Port Map

| Module Name        | Application Name   | Port        | Database          | Primary Responsibility                                |
|--------------------|--------------------|-------------|-------------------|-------------------------------------------------------|
| service-registry   | SERVICE-REGISTRY   | 8761        | None              | Netflix Eureka Service Registry & Discovery Server    |
| api-gateway        | API-GATEWAY        | 8080        | None              | Spring Cloud Gateway, Dynamic Routing, CORS Preflight |
| auth-service       | AUTH-SERVICE       | 8081        | sms_auth_db       | User Registration, Login, BCrypt, JWT Generation      |
| student-service    | STUDENT-SERVICE    | 8082        | sms_student_db    | Student CRUD, Excel Import/Export (Apache POI)        |
| academic-service   | ACADEMIC-SERVICE   | 8083        | sms_academic_db   | Classes, Subjects, Exams, Marks, Homework             |
| attendance-service | ATTENDANCE-SERVICE | 8084        | sms_attendance_db | Daily Attendance, Holidays, Student Leave Requests    |
| fee-service        | FEE-SERVICE        | 8085        | sms_fee_db        | Fee Invoices, Installment Payments, Status Tracking   |
| school-common      | (Shared JAR)       | —           | None              | Shared Enums, DTOs ( ApiResponse ), Common Exceptions |
| school-frontend    | (Vite React)       | 5173 / 5175 | LocalStorage      | Role-based Web Portal (Admin, Teacher, Parent)        |

1.2 Communication Technology Audit Checklist

- @FeignClient / OpenFeign: Not currently used in this project.
- RestClient (Spring Boot 3): Not currently used in this project.
- WebClient (Spring WebFlux): Used internally within api-gateway for reactive route forwarding, but not used in business services for inter-service communication.
- RestTemplate : Not currently used in this project.
- Direct Service-to-Service HTTP: Not currently used in this project.
- Apache Kafka / RabbitMQ: Not currently used in this project.
- Eureka Service Discovery: Actively used ( spring-cloud-starter-netflix-eureka-client on all services; spring-cloud-starter-netflix-eureka-server on service-registry ).
- Spring Cloud Gateway: Actively used ( lb:// dynamic URI resolution via Eureka).
- Axios: Actively used in school-frontend with a centralized interceptor targeting http://localhost:8080 .
- JWT (JSON Web Tokens): Actively used using io.jsonwebtoken:jjwt-api:0.11.5 with shared HMAC-SHA256 secret.

Step 2 — Real Architecture Diagram

In this project, microservices are **strictly decoupled** and do not query each other directly over the network. Instead, the project uses **Client-Side Orchestration / Gateway Routing** paired with **Stateless JWT Claims Propagation**.

2.1 Actual Architecture Diagram



2.2 How Cross-Service Data Correlation Works Without Feign

Since services do not make remote calls to each other: 1. **Shared Key References:** Services store plain numerical identifiers (e.g. `studentId: Long` , `parentId: Long` , `examId: Long` ). There are **no cross-database JPA foreign keys**. 2. **JWT Claims Propagation:** When a parent logs in, `auth-service` places `userId` , `role` , and `studentId` into the token payload. Downstream services ( `student-service` , `attendance-service` ) parse the token using a local `JwtDecoder` without calling `auth-service` . 3. **Frontend Aggregation:** When a unified view is needed (e.g., Parent Dashboard showing student name, attendance, fees, marks, homework), React fires parallel requests through the API Gateway, and the browser aggregates the JSON responses.

Step 3 — Eureka Service Registry & Discovery

3.1 What is Eureka and Why Do We Use It?

In a microservices architecture, services may run on dynamic IP addresses and ephemeral ports. Instead of hardcoding URLs like `http://localhost:8082` inside the API Gateway or client applications: - **Service Registry ( `service-registry` ):** Acts as the "phonebook" or central directory where every service registers its network location ( `ip-address` + `port` ). - **Service Discovery:** Allows clients (like `api-gateway` ) to ask Eureka: "Where is `STUDENT-SERVICE` currently running?" and receive the active IP and port.

### 3.2 Service Registry Configuration ( `service-registry/src/main/resources/application.yml` )

```
server:
  port: 8761

spring:
  application:
    name: SERVICE-REGISTRY

eureka:
  instance:
    hostname: localhost
  client:
    # Eureka server does NOT register itself
    register-with-eureka: false
    fetch-registry: false
    service-url:
      defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/
  server:
    wait-time-in-ms-when-sync-empty: 0
    response-cache-update-interval-ms: 3000
    eviction-interval-timer-in-ms: 5000
    enable-self-preservation: false
```

**Explanation of Key Lines:** - `register-with-eureka: false` : The Eureka server is the directory itself; it does not need to register as a client. - `fetch-registry: false` : The Eureka server does not need to fetch registry updates from anyone else. - `enable-self-preservation: false` : In local development, if network heartbeats are paused, Eureka aggressively evicts dead instances rather than keeping stale instances alive. - `eviction-interval-timer-in-ms: 5000` : Scans for dead microservices every 5 seconds.

### 3.3 How Microservices Register (e.g., `student-service/src/main/resources/application.yml` )

```
server:
  port: 8082

spring:
  application:
    name: STUDENT-SERVICE

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
    fetch-registry: true
    register-with-eureka: true
  instance:
    prefer-ip-address: true
    ip-address: 127.0.0.1
    hostname: localhost
    lease-renewal-interval-in-seconds: 5
    lease-expiration-duration-in-seconds: 10
```

**Line-by-Line Breakdown:** 1. `spring.application.name: STUDENT-SERVICE` : Defines the VIP (Virtual IP / Service Name) in Eureka. Always registered in uppercase. 2. `defaultZone: http://localhost:8761/eureka/` : The HTTP endpoint of the Eureka server to send registration and heartbeats. 3. `prefer-ip-address: true` & `ip-address: 127.0.0.1` : **Critical local development setting.** Forces Eureka to register the loopback IP `127.0.0.1` rather than an unreachable WiFi or VPN IP. 4. `lease-renewal-interval-in-seconds: 5` : The microservice sends a heartbeat ping to Eureka every 5 seconds to announce it is alive. 5. `lease-expiration-duration-in-seconds: 10` : If Eureka receives no heartbeat for 10 seconds, it marks this instance as down and removes it.

### 3.4 What Happens When a Service Goes Down?

1. The process terminates or crashes.
2. Heartbeats stop arriving at `http://localhost:8761/eureka/apps/STUDENT-SERVICE`.
3. After 10 seconds ( `lease-expiration-duration` ), Eureka evicts `STUDENT-SERVICE` from its cache.
4. The API Gateway fetches the updated registry during its 5-second fetch interval ( `registry-fetch-interval-seconds: 5` ) and stops routing requests to that dead port.
5. When a client requests `/api/students/**`, the Gateway immediately returns `503 Service Unavailable` or `500 Server Error` instead of hanging indefinitely.

## Step 4 — Spring Cloud API Gateway Deep Dive

### 4.1 Why the Frontend Calls the Gateway (:8080) and Not Individual Services

If React made direct calls to individual ports: - React would have to track 5 different ports ( `8081` , `8082` , `8083` , `8084` , `8085` ). - Every single microservice would have to configure and maintain CORS headers. - If a service changed port, the frontend code would break. - Authentication tokens would have to be handled across disparate endpoints without centralized entry logging.

By routing everything through `http://localhost:8080` , the API Gateway serves as a single entry point (Reverse Proxy).

### 4.2 Gateway Route Configuration ( `api-gateway/src/main/resources/application.yml` )

```
server:
  port: 8080

spring:
  application:
    name: API-GATEWAY
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
      routes:
        - id: auth-service
          uri: lb://AUTH-SERVICE
          predicates:
            - Path=/api/auth/**

        - id: student-service
          uri: lb://STUDENT-SERVICE
          predicates:
            - Path=/api/students/**

        - id: academic-service
          uri: lb://ACADEMIC-SERVICE
          predicates:
            - Path=/api/academic/**

        - id: attendance-service
          uri: lb://ATTENDANCE-SERVICE
          predicates:
            - Path=/api/attendance/**

        - id: fee-service
          uri: lb://FEE-SERVICE
          predicates:
            - Path=/api/fees/**

  httpclient:
    connect-timeout: 3000
    response-timeout: 10s
```

### 4.3 Understanding the `lb://` Prefix

- When a request matches `Path=/api/students/**` , the Gateway sees `uri: lb://STUDENT-SERVICE` .
- `lb://` stands for **Load Balancer**.
- Instead of connecting to a static IP, Spring Cloud Gateway consults its local copy of the Eureka service registry.
- It finds all running instances registered under the name `STUDENT-SERVICE` (e.g. `127.0.0.1:8082` ) and balances incoming traffic across them.

### 4.4 Gateway CORS Configuration ( `CorsConfig.java` )

Because React runs on `http://localhost:5173` (or `5175` ) and API Gateway runs on `http://localhost:8080` , browsers send an `OPTIONS` HTTP preflight request before making `POST` , `PUT` , or authenticated calls.

In `com.school.gateway.config.CorsConfig` :

```
@Configuration
public class CorsConfig {

    @Bean
    public CorsWebFilter corsWebFilter() {
        CorsConfiguration corsConfig = new CorsConfiguration();
        corsConfig.setAllowedOriginPatterns(List.of(
            "http://localhost:*",
            "http://127.0.0.1:*",
            "http://[::1]:*"
        ));
        corsConfig.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH", "HEAD"));
        corsConfig.setAllowedHeaders(List.of("*"));
        corsConfig.setExposedHeaders(List.of("*"));
        corsConfig.setAllowCredentials(true);
        corsConfig.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", corsConfig);

        return new CorsWebFilter(source);
    }
}
```

`CorsWebFilter` intercepts the browser's preflight request and responds with HTTP `200 OK` and the appropriate `Access-Control-Allow-Origin` and `Access-Control-Allow-Headers` headers.

## Step 5 — Inter-Service Communication Audit & Cross-Service Pattern

### 5.1 The Ground Truth Audit

A deep search was conducted across all Java source files and Maven POMs for: - `@FeignClient` - `RestTemplate` - `RestClient` - `WebClient` - `HttpClient` / `HttpURLConnection` - Kafka / RabbitMQ

Result:

No direct synchronous or asynchronous microservice-to-microservice network calls exist in this codebase.

### 5.2 Why Does the Project Work Perfectly Without Service-to-Service Calls?

In monolithic systems or naive microservice setups, one service frequently calls another: `academic-service` —(HTTP)—> `student-service` —(HTTP)—> `auth-service`

This creates **tight coupling, high network latency, cascading failures, and distributed transaction complexity**.

In this project, the architecture uses a cleaner, more resilient design: 1. **Decoupled Bounded Contexts**: Each microservice is responsible solely for its own domain table and data: - `student-service` manages student demographic details. - `academic-service` manages grades, subjects, and exams. - `attendance-service` manages attendance dates and leaves. - `fee-service` manages financial ledgers and invoices. 2. **Identifier Correlation**: When `academic-service` records a mark, it simply stores `studentId: Long`. It does not need to know the student's home address or birth date. 3. **Stateless JWT Claims**: The user identity and child mapping ( `studentId` ) are encoded into the JWT when the user logs in. Every downstream microservice reads this information directly from the token. 4. **Client-Side Aggregation**: When a user interface requires composite data, the React client queries the respective microservices through the Gateway in parallel using `Promise.allSettled()`.

## Step 6 — OpenFeign Audit

### Status in Current Project

"Not currently used in this project."

### Educational Comparison: What is OpenFeign?

Spring Cloud OpenFeign is a declarative HTTP web service client. Instead of writing boilerplate HTTP code, a developer defines an interface and annotates it:

```
// Example for learning only – NOT PRESENT IN CURRENT PROJECT:
@FeignClient(name = "STUDENT-SERVICE")
public interface StudentClient {

    @GetMapping("/api/students/{id}")
    StudentDTO getStudentById(@PathVariable("id") Long id);

}
```

#### How OpenFeign Works Under the Hood:

1. At startup, Spring scans for `@EnableFeignClients` and `@FeignClient`.
2. Spring creates a dynamic JDK runtime proxy implementation of the interface.
3. When `studentClient.getStudentById(1L)` is invoked, OpenFeign queries Eureka for `STUDENT-SERVICE`, constructs an HTTP `GET /api/students/1`, executes the request, deserializes the JSON response into `StudentDTO`, and returns it.
4. If target service is unavailable, OpenFeign throws a `FeignException` unless a fallback factory is configured.

**Why this project does not use it:** OpenFeign adds unnecessary complexity for a system whose domain models are cleanly segregated and aggregated via the frontend.

## Step 7 — RestClient Audit

### Status in Current Project

"Not currently used in this project."

### Educational Comparison: What is RestClient?

Introduced in **Spring Boot 3.2 / Spring Framework 6.1**, `RestClient` is a modern, synchronous HTTP client that provides a fluent, functional API similar to `WebClient`, but designed for synchronous execution on standard non-reactive servlets.

```
// Example for learning only – NOT PRESENT IN CURRENT PROJECT:
RestClient restClient = RestClient.builder()
    .baseUrl("http://localhost:8082")
    .build();

StudentDTO student = restClient.get()
    .uri("/api/students/{id}", 1L)
    .retrieve()
    .body(StudentDTO.class);
```

**Why this project does not use it:** There is no requirement in this architecture for one backend service to invoke another during transaction execution.

## Step 8 — WebClient Audit

### Status in Current Project

"WebClient is not currently used for business service-to-service communication in this project."

### Nuance: WebFlux Runtime in API Gateway

The `api-gateway` module depends on `spring-cloud-starter-gateway`, which runs on a reactive **Project Reactor / Netty** engine (not Tomcat). Gateway internally uses reactive HTTP dispatching to forward packets to downstream services, but **no custom `WebClient` bean is injected into any business service** (`student-service`, `auth-service`, etc.).

## Step 9 — Distributed JWT Authentication & Security

The system employs **Stateless Distributed Token Authentication** with a symmetric shared secret.

## 9.1 The Step-by-Step Flow

1. User enters email & password on React Login page.
2. React sends POST `http://localhost:8080/api/auth/login`.
3. API Gateway matches `/api/auth/**` and routes to AUTH-SERVICE (:8081).
4. AuthController passes credentials to `UserService.login()`.
5. UserService checks email against PostgreSQL database (`sms_auth_db`).
6. Password verified using `BCryptPasswordEncoder.matches()`.
7. `JwtUtil.generateToken()` generates HMAC-SHA256 token embedding:
  - subject: email
  - claim("userId", user.getId())
  - claim("role", user.getRole().name())
  - claim("studentId", user.getStudentId()) <-- populated if PARENT
8. AuthResponse returns { token, type: "Bearer", userId, name, email, role, studentId }.
9. React saves token into `localStorage.setItem('token', token)`.
10. Axios Request Interceptor in `api.js` automatically adds header to ALL subsequent requests:
 

```
Authorization: Bearer <token>
```
11. When request is sent to `http://localhost:8080/api/attendance/leave`:
  - Gateway forwards request with Authorization header to ATTENDANCE-SERVICE (:8084).
  - AttendanceController calls `jwtDecoder.extractRaw(authHeader)`.
  - JwtDecoder validates signature locally using the shared secret.
  - AttendanceController extracts `userId` and `studentId` directly from token claims!

## 9.2 The Shared Secret Secret Key

In `auth-service`, `student-service`, and `attendance-service` `application.yml` :

```
jwt:
  secret: ${JWT_SECRET:SchoolManagementSystemSecretKey2024SuperSecretKeyForJWTTokenGeneration}
  expiration: 86400000 # 24 hours in ms
```

Because both services share the exact same 256-bit cryptographic secret, `student-service` and `attendance-service` can verify the signature and trust the claims without calling `auth-service` !

## 9.3 Inspection of `JwtUtil.java` (in `auth-service`)

```
public String generateToken(String email, String role, Long userId, Long studentId) {
    var builder = Jwts.builder()
        .setSubject(email)
        .claim("role", role)
        .claim("userId", userId)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + jwtExpiration));

    if (studentId != null) {
        builder.claim("studentId", studentId);
    }

    return builder.signWith(getSigningKey(), SignatureAlgorithm.HS256).compact();
}
```

## 9.4 Inspection of JwtDecoder.java (in student-service & attendance-service)

```
@Component
public class JwtDecoder {

    @Value("${jwt.secret}")
    private String jwtSecret;

    private Key getSigningKey() {
        return Keys.hmacShaKeyFor(jwtSecret.getBytes());
    }

    public boolean isValidToken(String token) {
        if (!StringUtils.hasText(token)) return false;
        try {
            Jwts.parserBuilder().setSigningKey(getSigningKey()).build().parseClaimsJws(token);
            return true;
        } catch (JwtException e) {
            return false;
        }
    }

    public Long getStudentId(String token) {
        Claims claims = Jwts.parserBuilder()
            .setSigningKey(getSigningKey()).build()
            .parseClaimsJws(token).getBody();
        return claims.get("studentId", Long.class);
    }

    public Long getUserId(String token) {
        Claims claims = Jwts.parserBuilder()
            .setSigningKey(getSigningKey()).build()
            .parseClaimsJws(token).getBody();
        return claims.get("userId", Long.class);
    }

    public String extractRaw(String authHeader) {
        if (StringUtils.hasText(authHeader) && authHeader.startsWith("Bearer ")) {
            return authHeader.substring(7);
        }
        return null;
    }
}
```

## Step 10 — Parent → Student Data & Security Flow

One of the most critical aspects of this project is how a Parent user is associated with their child and how security is enforced.

### 10.1 How Parent is Linked to Student in the Database

In `auth-service/src/main/java/com/school/auth/entity/User.java`:

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role; // ADMIN, TEACHER, PARENT

    @Column(name = "student_id")
    private Long studentId;
}
```

**Architecture Rule Illustrated Here:** Notice that `studentId` is a plain `Long` column. It is **not** a JPA `@ManyToOne` or `@JoinColumn` to `Student`. Why? Because `Student` is in a completely different database (`sms_student_db`), managed by a different service (`student-service`).

## 10.2 Admin Creates Parent Account

Admin calls `POST /api/auth/parents` :

```
{
  "name": "David Miller",
  "email": "david.miller@example.com",
  "password": "Password@123",
  "studentId": 14
}
```

`UserService.createParent()` validates that `studentId != null`, encodes the password, sets `role = Role.PARENT`, and saves the user record to `sms_auth_db`.

## 10.3 Parent Login & Dashboard Data Loading

1. Parent logs in. The token returned contains `studentId: 14`.
2. In `ParentDashboard.jsx` :

```
const { user } = useAuth();
const studentId = user?.studentId;

const loadAllData = async () => {
  // Step 1: Fetch child profile from student-service
  const stuRes = await studentService.getStudentById(studentId);
  setChild(stuRes.data);

  // Step 2: Concurrently fetch attendance, fees, marks, homework, holidays, leaves
  const [attRes, feeRes, mrkRes, hwRes, holRes, lvRes] = await Promise.allSettled([
    attendanceService.getAttendanceByStudent(studentId),
    feeService.getFeesByStudent(studentId),
    academicService.getMarksByStudent(studentId),
    homeworkService.getByClass(stuRes.data.className),
    holidayService.getAll(),
    leaveService.getMyLeave(),
  ]);
};
```

## 10.4 Enforcing Ownership & Preventing Unauthorized Data Access

What prevents Parent A from viewing Parent B's child data or submitting leave for someone else? 1. In `StudentController.java` ( `getStudentsByParent` ):

```
if ("PARENT".equals(role) && !parentId.equals(callerUserId)) {
  return ResponseEntity.status(HttpStatus.FORBIDDEN)
    .body(ApiResponse.error("Forbidden: you can only view your own children"));
}
```

1. In `AttendanceController.java` ( `submitLeave` ):

```
// Notice: studentId is NEVER accepted from the request body!
// It is read straight from the cryptographically verified JWT token:
Long studentId = jwtDecoder.getStudentId(token);
Long parentId = jwtDecoder.getUserId(token);

LeaveRequest saved = attendanceService.submitLeave(studentId, parentId, request);
```

Even if an attacker modifies the request JSON payload, the backend ignores it and uses the token's embedded `studentId`.

# Step 11 — Teacher → Academic & Assessment Flow

## 11.1 The Workflow

A teacher needs to grade students across exams and subjects.

```

Teacher Dashboard
↓
1. Loads all students: studentService.getAllStudents() -> STUDENT-SERVICE (:8082)
2. Loads all exams: academicService.getAllExams() -> ACADEMIC-SERVICE (:8083)
3. Loads all subjects: academicService.getAllSubjects() -> ACADEMIC-SERVICE (:8083)
↓
Teacher selects Exam #1 ("Midterm Exam") & Subject #2 ("Mathematics")
↓
Teacher inputs marks for each student into marksMap
↓
Teacher clicks "Submit Evaluation Marks"
↓
academicService.submitMarks({ studentId: 5, examId: 1, subjectId: 2, marksObtained: 88, maxMarks: 100 })
↓
API Gateway routes POST /api/academic/marks to ACADEMIC-SERVICE (:8083)
↓
AcademicController.createMark() calls AcademicService.createMark()
↓
Saved to marks table in sms_academic_db
↓
When Parent logs in, ParentDashboard calls GET /api/academic/marks/student/5
↓
Marks appear immediately in Parent portal!

```

## 11.2 Entity Model ( `academic-service/.../Mark.java` )

```

@Entity
@Table(name = "marks")
@Data
public class Mark {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Long studentId; // Plain Long reference to student-service

    @Column(nullable = false)
    private Long examId;

    private Integer marksObtained;
    private String grade;
    private String remarks;
}

```

# Step 12 — Fee & Payment Lifecycle Flow

## 12.1 The Fee Entities ( `fee-service` )

The `fee-service` handles tuition, transport, library, and examination fees: - **Fee Entity** ( `sms_fee_db.fees` ): Holds `studentId`, `totalAmount`, `paidAmount`, `pendingAmount`, `status` ( `PAID`, `PARTIAL`, `UNPAID` ), `dueDate`, and `feeType`. - **Payment Entity** ( `sms_fee_db.payments` ): Holds `feeId`, `amountPaid`, `paymentDate`, `paymentMethod` ( `CASH`, `ONLINE`, `BANK_TRANSFER`, `CHEQUE` ), and `note`.

## 12.2 Admin Fee Creation Flow

- Admin selects a student on `FeesPage.jsx`.
- Admin clicks "Generate Fee Invoice" (e.g., Tuition: \$1,200).
- React calls `feeService.createFee(...)`.
- Gateway routes `POST /api/fees` to `FEE-SERVICE (:8085)`.
- `FeeService` computes:
  - `paidAmount = 0.00`
  - `pendingAmount = totalAmount - paidAmount = 1200.00`
  - `status = UNPAID`
- Persisted to `sms_fee_db`.

## 12.3 Payment Recording Flow

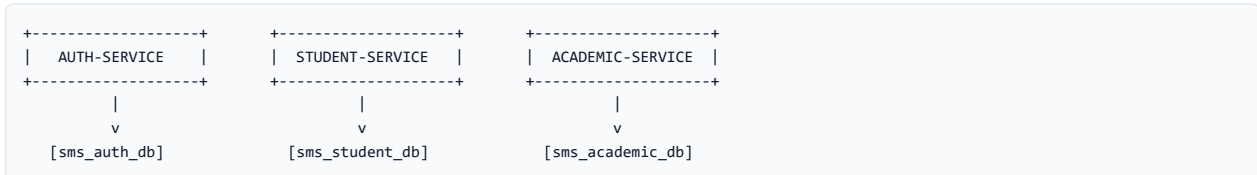
- Admin records payment of \$400.
- React calls `feeService.recordPayment(feeId, { amountPaid: 400, paymentMethod: 'ONLINE' })`.
- `POST /api/fees/{feeId}/payments` reaches `FeeController.recordPayment()`.
- `FeeService.recordPayment()` executes transaction:
- Creates new `Payment` record with `amountPaid = 400.00`.

6. Fetches parent `Fee` record.
7. Updates `fee.paidAmount = fee.paidAmount + 400 = 400.00`.
8. Updates `fee.pendingAmount = fee.totalAmount - fee.paidAmount = 800.00`.
9. Updates `status = FeeStatus.PARTIAL`.
10. Saves both records to `sms_fee_db`.
11. Parent opening `ParentDashboard.jsx` sees outstanding dues reduced to \$800.00 in real-time.

## Step 13 — Database Ownership (Database-per-Service)

### 13.1 Microservices Golden Rule: Private Databases

Each service possesses exclusive ownership of its database:



### 13.2 Why Direct Cross-Database Access is Prohibited

Beginners often ask: "Why can't `academic-service` just run a SQL JOIN on `sms_student_db.students`?"

**Reasons why this breaks microservices architecture:**

1. **Schema Coupling:** If `student-service` renames column `name` to `first_name` and `last_name`, `academic-service` queries crash silently.
2. **Security Vulnerability:** Bypasses domain validation, business logic, and security rules enforced in the owning service.
3. **Connection Pool Contention:** A slow query in `academic-service` can exhaust connection pool resources for `student-service`.
4. **Independent Scalability & Cloud Deployment:** In production, `sms_student_db` might be hosted in AWS RDS US-East while `sms_academic_db` might be scaled on a separate database cluster.

### 13.3 How This Project Implements Database Independence

1. Each service defines its own `spring.datasource.url` in `application.yml` targeting a separate database name.
2. Entities use standard JPA `@Id` values ( `Long studentId`, `Long parentId`, `Long examId` ).
3. Cross-service domain joins are replaced with **Client-Side Aggregation** in React.

## Step 14 — Complete Real Request Traces

Here are 3 complete, line-by-line real request traces through the actual classes of this project:

### Trace 1: Parent Opens Assessment / Marks Tab

**Goal:** Parent with `studentId = 14` views assessment grades.

| Step | Component         | Class / File            | Method / Route                                                       | Action Performed                                                                                   |
|------|-------------------|-------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1    | Browser / UI      | ParentDashboard.jsx     | setActiveTab('marks')                                                | User selects Marks tab.                                                                            |
| 2    | Frontend Service  | academicService.js      | getMarksByStudent(14)                                                | Executes <code>api.get('/api/academic/marks/student/14')</code>                                    |
| 3    | Axios Interceptor | api.js                  | Request Interceptor                                                  | Appends <code>Authorization: Bearer &lt;token&gt;</code> from <code>localStorage</code> .          |
| 4    | Network Call      | Browser Network Engine  | GET <code>http://localhost:8080/api/academic/marks/student/14</code> | HTTP request sent to Gateway port <code>8080</code> .                                              |
| 5    | API Gateway       | application.yml         | Route Match                                                          | Matches <code>- id: academic-service , Path=/api/academic/**</code> .                              |
| 6    | Load Balancer     | Spring Cloud Gateway    | <code>lb://ACADEMIC-SERVICE</code>                                   | Gateway queries Eureka for <code>ACADEMIC-SERVICE</code> , finds <code>127.0.0.1:8083</code> .     |
| 7    | Reverse Proxy     | Netty HTTP Dispatcher   | Forward Request                                                      | Forwards request and <code>Authorization</code> header to <code>127.0.0.1:8083</code> .            |
| 8    | Target Controller | AcademicController.java | <code>getMarksByStudent(@PathVariable Long studentId)</code>         | Endpoint <code>@GetMapping("/marks/student/{studentId}")</code> triggered.                         |
| 9    | Business Service  | AcademicService.java    | <code>getMarksByStudent(14L)</code>                                  | Executes <code>markRepository.findById(14L)</code> .                                               |
| 10   | Data Layer        | MarkRepository.java     | Spring Data JPA query                                                | Runs <code>SELECT * FROM marks WHERE student_id = 14</code> against <code>sms_academic_db</code> . |
| 11   | Response Return   | AcademicController.java | <code>ResponseEntity.ok(ApiResponse.success(marks))</code>           | Returns HTTP 200 with JSON payload containing array of marks.                                      |
| 12   | Gateway Relay     | API Gateway             | Reactive Bridge                                                      | Pipes HTTP 200 response back to frontend.                                                          |
| 13   | React State       | ParentDashboard.jsx     | <code>setMarks(mrkRes.value.data)</code>                             | Updates state and renders grades table in UI.                                                      |

## Trace 2: Teacher Submits Student Evaluation Marks

**Goal:** Teacher enters mark `92.5` for Student #7 in Exam #2, Subject #3.

| Step | Component         | Class / File            | Method / Route                                    | Action Performed                                                                              |
|------|-------------------|-------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1    | Browser / UI      | TeacherDashboard.jsx    | handleSubmitMarks()                               | Teacher clicks "Submit Evaluation Marks".                                                     |
| 2    | Frontend Service  | academicService.js      | submitMarks(payload)                              | Calls <code>api.post('/api/academic/marks', payload)</code> .                                 |
| 3    | Payload           | TeacherDashboard.jsx    | JSON Body                                         | <code>{"studentId":7, "examId":2, "subjectId":3, "marksObtained":92.5, "maxMarks":100}</code> |
| 4    | Axios Interceptor | api.js                  | Request Interceptor                               | Attaches <code>Authorization: Bearer &lt;jwt_teacher&gt;</code> header.                       |
| 5    | Gateway Routing   | application.yml         | Predicate Filter                                  | Matches route <code>academic-service -&gt; lb://ACADEMIC-SERVICE</code> .                     |
| 6    | Target Controller | AcademicController.java | <code>createMark(@RequestBody Mark mark)</code>   | Method handles <code>POST /api/academic/marks</code> .                                        |
| 7    | Business Logic    | AcademicService.java    | <code>createMark(Mark mark)</code>                | Calculates grade: <code>if (pct &gt;= 90) mark.setGrade("A+");</code> .                       |
| 8    | Database Write    | MarkRepository.java     | <code>markRepository.save(mark)</code>            | Executes SQL <code>INSERT INTO marks ...</code> in <code>sms_academic_db</code> .             |
| 9    | Success Response  | AcademicController.java | <code>ResponseEntity.status(201).body(...)</code> | Returns <code>ApiResponse.success("Mark recorded", created)</code> .                          |
| 10   | UI Notification   | TeacherDashboard.jsx    | <code>addToast(..., 'success')</code>             | Displays success toast notification on screen.                                                |

## Trace 3: Parent Submits Student Leave Request

**Goal:** Parent submits medical leave for their child without exposing or passing `studentId` in the request body.

| Step | Component                    | Class / File              | Method / Route                         | Action Performed                                                                     |
|------|------------------------------|---------------------------|----------------------------------------|--------------------------------------------------------------------------------------|
| 1    | Browser / UI                 | ParentDashboard.jsx       | handleSubmitLeave()                    | Parent fills start date, end date, reason and clicks Submit.                         |
| 2    | Frontend Service             | leaveService.js           | submit(leaveForm)                      | Calls <code>api.post('/api/attendance/leave', leaveForm)</code> .                    |
| 3    | Axios Interceptor            | api.js                    | Request Interceptor                    | Injects parent JWT token.                                                            |
| 4    | API Gateway                  | application.yml           | Dynamic Route                          | Matches <code>/api/attendance/**</code> -> <code>lb://ATTENDANCE-SERVICE</code> .    |
| 5    | Gateway Forward              | Netty Client              | 127.0.0.1:8084                         | Sends <code>POST /api/attendance/leave</code> to Attendance Service.                 |
| 6    | Controller Header Inspection | AttendanceController.java | submitLeave(...)                       | Reads <code>request.getHeader("Authorization")</code> .                              |
| 7    | JWT Token Extraction         | JwtDecoder.java           | extractRaw(authHeader)                 | Strips <code>"Bearer "</code> prefix.                                                |
| 8    | Signature Validation         | JwtDecoder.java           | isValidToken(token)                    | Verifies token HMAC signature with shared secret.                                    |
| 9    | Claim Extraction             | JwtDecoder.java           | getStudentId(token) & getUserId(token) | Extracts <code>studentId = 14</code> , <code>parentId = 3</code> from signed claims! |
| 10   | Security Assertion           | AttendanceController.java | Null validation                        | Validates that <code>studentId != null</code> (rejects non-parents).                 |
| 11   | Service Call                 | AttendanceService.java    | submitLeave(studentId, parentId, req)  | Sets <code>status = "PENDING"</code> , links to student 14.                          |
| 12   | Database Write               | LeaveRequestRepository    | save(leaveRequest)                     | Inserts into <code>sms_attendance_db.leave_requests</code> .                         |
| 13   | Response                     | AttendanceController.java | ResponseEntity.status(201)...          | Returns HTTP 201 Created to frontend.                                                |

## Step 15 — Core Code Snippets & Line-by-Line Breakdown

### 15.1 API Gateway Dynamic Route Definition

```
# Location: api-gateway/src/main/resources/application.yml
- id: student-service
  uri: lb://STUDENT-SERVICE
  predicates:
    - Path=/api/students/**
```

- `- id: student-service` : Unique identifier for this route within Spring Cloud Gateway.
- `uri: lb://STUDENT-SERVICE` : Instructs Spring Cloud LoadBalancer to look up `STUDENT-SERVICE` in Eureka and distribute traffic across available instances.
- `predicates: - Path=/api/students/**` : Any incoming HTTP request starting with `/api/students/` will be handled by this route.

### 15.2 Eureka Instance Registration

```
# Location: student-service/src/main/resources/application.yml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    ip-address: 127.0.0.1
    lease-renewal-interval-in-seconds: 5
```

- `defaultZone` : Points to the Eureka registry server.
- `prefer-ip-address: true` & `ip-address: 127.0.0.1` : Explicitly binds registration to the local loopback interface, eliminating IP mismatch errors on multi-homed or VPN machines.
- `lease-renewal-interval-in-seconds: 5` : Sends heartbeat every 5 seconds.

### 15.3 Axios Centralized Request Interceptor

```
// Location: school-frontend/src/services/api.js
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers['Authorization'] = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);
```

- Intercepts every outgoing HTTP call initiated by `api.get()` or `api.post()`.
- Fetches the current JWT from browser storage ( `localStorage` ).
- Inserts `Authorization: Bearer <token>` header so backend microservices can verify identity.

### 15.4 Stateless Token Decoding in Downstream Services

```
// Location: student-service/src/main/java/com/school/student/security/JwtDecoder.java
public Long getStudentId(String token) {
    Claims claims = Jwts.parserBuilder()
        .setSigningKey(getSigningKey()).build()
        .parseClaimsJws(token).getBody();
    return claims.get("studentId", Long.class);
}
```

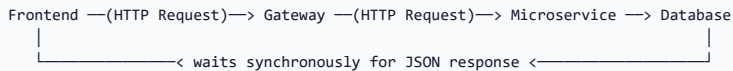
- `Jwts.parserBuilder().setSigningKey(...)` : Reconstructs parser with the shared 256-bit secret key.
- `.parseClaimsJws(token)` : Validates signature and expiration date. Throws `JwtException` if tampered with.
- `claims.get("studentId", Long.class)` : Directly extracts the student identifier embedded by `auth-service`.

## Step 16 — Communication Methods Comparison Table

| Communication Method           | Used in This Project? | Role / Implementation in Current Project                                                                              | Industry Pros                                                           | Industry Cons                                                     |
|--------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>Spring Cloud Gateway</b>    | YES                   | Entry reverse-proxy on port <code>8080</code> ; handles CORS preflight and dynamic <code>lb://</code> routing.        | Centralized entry point, hides backend topologies, manages CORS.        | Single point of failure if not clustered.                         |
| <b>Netflix Eureka</b>          | YES                   | Service registry on port <code>8761</code> ; tracks microservice instances via heartbeats on <code>127.0.0.1</code> . | Automatic IP/port discovery, dynamic scaling, zero hardcoded endpoints. | Eventual consistency lag during fast restarts.                    |
| <b>Axios HTTP Client</b>       | YES                   | React frontend client with base URL <code>http://localhost:8080</code> and token interceptor.                         | Easy interceptors, automatic JSON transformation, browser standard.     | Limited to browser runtime.                                       |
| <b>OpenFeign</b>               | NO                    | <i>Not currently used in this project.</i>                                                                            | Declarative Java HTTP client, integrates with Ribbon/LoadBalancer.      | Adds interface overhead; unnecessary when services are decoupled. |
| <b>RestClient</b>              | NO                    | <i>Not currently used in this project.</i>                                                                            | Synchronous fluent HTTP client in Spring Boot 3.                        | Still creates synchronous coupling between backends.              |
| <b>WebClient</b>               | NO<br>(Business)      | Internal to <code>api-gateway</code> reactive stack, but not in business logic.                                       | Non-blocking, reactive, handles high concurrent throughput.             | Reactive programming complexity (Mono/Flux).                      |
| <b>RestTemplate</b>            | NO                    | <i>Not currently used in this project.</i>                                                                            | Simple classic Spring HTTP client.                                      | Maintenance mode in Spring 6 in favor of RestClient.              |
| <b>Apache Kafka / RabbitMQ</b> | NO                    | <i>Not currently used in this project.</i>                                                                            | High throughput asynchronous event streaming, total decoupling.         | Requires ZooKeeper/KRaft broker setup, heavy memory footprint.    |

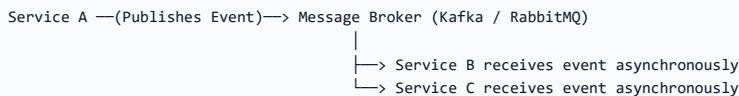
## Step 17 — Synchronous vs. Asynchronous Communication

### 17.1 How Synchronous Communication Works (This Project's Architecture)



- **Nature:** Request / Response.
- **Protocol:** HTTP 1.1 / REST JSON.
- **Characteristics:** The calling client sends an HTTP request and blocks or waits until the server completes processing and returns an HTTP status code (200, 201, 400, etc.).
- **Why this project uses it:** Simple to debug, straightforward to understand, aligns with CRUD web applications, zero extra infrastructure required.

### 17.2 How Asynchronous Communication Works (Not Implemented)



- **Status in this project:** Not currently implemented.
- In an asynchronous architecture, Service A emits an event (e.g. `StudentEnrolledEvent`) onto a message broker topic and returns immediately without waiting for Service B or Service C to process it.

## Step 18 — Failure Scenarios & Resilience Audit

| Failure Scenario                              | Current Implemented Behavior                                                                                                                                                                                                                                      | Recommended Production Improvement                                                                                      |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Eureka Server Goes Down</b>                | Microservices and Gateway continue functioning using their locally cached registry entries for a short time. However, new service instances cannot register, and dead instances are not evicted.                                                                  | Deploy Eureka in High Availability (HA) mode with peer-to-peer replica clusters.                                        |
| <b>API Gateway Goes Down</b>                  | React frontend receives <code>Network Error</code> ( <code>ERR_CONNECTION_REFUSED</code> ) on all <code>/api/**</code> calls. Frontend redirects to login or displays error toasts.                                                                               | Run multiple Gateway instances behind an infrastructure load balancer (like NGINX, AWS ALB, or HAProxy).                |
| <b>Student Service Goes Down</b>              | Gateway returns <code>500 Server Error</code> or <code>503 Service Unavailable</code> . In <code>ParentDashboard</code> , <code>Promise.allSettled()</code> prevents total crash: other widgets (Holidays, Fees) still load, but Child Profile displays an error. | Implement <b>Resilience4j Circuit Breaker</b> and Fallback in Gateway or frontend to serve cached profiles.             |
| <b>PostgreSQL Database Goes Down</b>          | Spring Boot throws <code>CannotCreateTransactionException</code> / HikariCP connection timeout. Global exception handler returns HTTP 500 error response.                                                                                                         | Configure database read-replicas, connection pool health monitoring, and automatic RDS failover.                        |
| <b>JWT is Expired (&gt; 24 Hours)</b>         | Downstream <code>JwtDecoder</code> or <code>JwtAuthFilter</code> throws <code>ExpiredJwtException</code> . Returns HTTP 401 Unauthorized. Axios response interceptor intercepts 401, clears localStorage, and redirects to <code>/login</code> .                  | Implement <b>Refresh Token Rotation</b> (short-lived access tokens of 15m + long-lived secure HttpOnly refresh tokens). |
| <b>Parent Accesses Another Student's Data</b> | <code>StudentController</code> checks <code>parentId.equals(callerUserId)</code> . If mismatch, returns HTTP 403 Forbidden. In <code>AttendanceController</code> , <code>studentId</code> is read exclusively from JWT claims, making tampering impossible.       | Current implementation is secure. Audit logging can be added to flag suspicious requests.                               |
| <b>Gateway Request Times Out</b>              | Gateway has <code>connect-timeout: 3000ms</code> and <code>response-timeout: 10s</code> . If a microservice hangs, Gateway returns HTTP 504 Gateway Timeout after 10 seconds.                                                                                     | Implement per-route timeouts and graceful fallback degradation using Resilience4j.                                      |

## Step 19 — Top 10 Beginner Microservices Mistakes

### 1. Direct Cross-Service Database Access

- **The Mistake:** Connecting `fee-service` directly to `sms_student_db` to read student names.
- **Why it happens:** Beginners want to write simple SQL `JOIN` queries.
- **The Remedy:** Maintain private databases. Have the frontend aggregate data or pass necessary IDs via APIs.

### 2. Hardcoding Localhost URLs in the Frontend

- **The Mistake:** Writing `axios.get("http://localhost:8082/api/students")` inside a React component.
- **Why it happens:** The developer wants quick tests during local development.

- **The Remedy:** Always target the API Gateway ( `http://localhost:8080/api/students` ).

### 3. Forgetting Eureka Loopback Binding ( `127.0.0.1` )

- **The Mistake:** Leaving `prefer-ip-address: false` on Windows machines with virtual adapters or WiFi.
- **Why it happens:** Eureka registers a non-routable host IP (e.g. `192.168.x.x` or `172.x.x.x` ), causing Gateway forwarding to time out.
- **The Remedy:** Explicitly configure `prefer-ip-address: true` and `ip-address: 127.0.0.1` in all microservice YAML files.

### 4. Case Sensitivity in Service Names

- **The Mistake:** Writing `uri: lb://student-service` in Gateway while the service registers as `STUDENT-SERVICE` .
- **Why it happens:** In Spring Cloud Gateway, Eureka VIP matching is case-sensitive unless `lower-case-service-id: true` is configured.
- **The Remedy:** Keep service names consistently uppercase (e.g. `STUDENT-SERVICE` ) and enable `lower-case-service-id: true` .

### 5. Missing CORS Configuration on the API Gateway

- **The Mistake:** React throws: *"CORS header 'Access-Control-Allow-Origin' missing"*.
- **Why it happens:** Modern browsers block cross-origin requests when frontend port ( `5173` ) differs from backend port ( `8080` ).
- **The Remedy:** Implement a reactive `CorsWebFilter` in `api-gateway` that permits required localhost origins.

### 6. Missing the `Authorization` Header in Service Calls

- **The Mistake:** A request succeeds in Postman without headers, but fails in the React application.
- **Why it happens:** The frontend developer forgot to configure an Axios request interceptor to inject `Bearer <token>` .
- **The Remedy:** Centralize Axios configuration in `api.js` and attach the token automatically on every outgoing request.

### 7. Over-Engineering Microservices with Unnecessary Tools

- **The Mistake:** Adding Kafka, Redis, Zipkin, Config Server, and Feign to a small 5-service school project before needing them.
- **Why it happens:** Copying enterprise templates without evaluating real project requirements.
- **The Remedy:** Keep architecture clean and simple. Solve immediate requirements first with robust REST APIs.

### 8. Exposing Internal Microservice Ports to the Public

- **The Mistake:** Opening ports `8081 – 8085` to the public internet.
- **Why it happens:** Lack of firewall or network security boundaries.
- **The Remedy:** In production, place microservices inside a private Virtual Private Cloud (VPC) subnet; only expose Gateway port `80` / `443` .

### 9. Lack of Request Timeouts

- **The Mistake:** A slow SQL query in one service causes the Gateway connection pool to hang forever.
- **Why it happens:** Default HTTP clients wait indefinitely without timeout parameters.
- **The Remedy:** Configure `connect-timeout` (e.g. 3000ms) and `response-timeout` (e.g. 10s) in the Gateway configuration.

### 10. Cyclic Dependencies

- **The Mistake:** Service A calls Service B, which calls Service C, which calls Service A.
- **Why it happens:** Poor domain boundary definitions.
- **The Remedy:** Follow unidirectional data flows and rely on event publication or frontend aggregation.

## Step 20 — Microservices Technical Interview Preparation

### Beginner Questions & Answers

#### Q1: What is a microservice?

**Answer:** A microservice is an architectural approach where an application is composed of small, independent services that communicate over lightweight protocols (such as HTTP REST). Each service focuses on a single business domain (e.g., student management, billing) and maintains its own private database.

#### Q2: What is the role of Netflix Eureka in this project?

**Answer:** Eureka acts as the Service Registry and Discovery server. Instead of hardcoding microservice IP addresses and ports inside the API Gateway, every microservice registers its address with Eureka at startup. The API Gateway queries Eureka dynamically to route incoming client traffic to the appropriate service instance.

#### Q3: What is an API Gateway and why is it needed?

**Answer:** The API Gateway (running on port `8080` ) is the single entry point for all frontend requests. It provides reverse proxy routing, load balancing, CORS preflight handling, and security filtering. It ensures the frontend does not need to know individual service ports.

**Q4: How does service discovery work when a request arrives at the Gateway?**

**Answer:** When React requests `http://localhost:8080/api/students`, the Gateway matches the route predicate `Path=/api/students/**`. The route URI is `lb://STUDENT-SERVICE`. The Gateway queries Eureka's registry table for `STUDENT-SERVICE`, retrieves its active address (`127.0.0.1:8082`), and dispatches the request.

**Q5: What is OpenFeign? Is it used in this project?**

**Answer:** OpenFeign is a declarative HTTP client library from Spring Cloud that simplifies calling other REST services using annotated Java interfaces. **It is not currently used in this project** because this system avoids direct backend-to-backend HTTP calls, utilizing frontend aggregation and stateless JWT claims instead.

**Q6: Why should microservices NOT share a database?**

**Answer:** Sharing databases creates tight coupling. If one service modifies a table schema, other services break. It also violates domain boundaries, bypasses service business validation rules, and causes database connection contention.

**Q7: How does JWT authentication work across multiple services in this system?**

**Answer:** Authentication is stateless. When a user logs in, `auth-service` generates a JWT containing the user's ID, role, and linked student ID, signed with an HMAC-SHA256 secret key. Other microservices share the same secret key and decode the token locally using `JwtDecoder`, eliminating the need to query `auth-service` over the network.

**Q8: What is the difference between client-side load balancing and server-side load balancing?**

**Answer:** In server-side load balancing (e.g. NGINX or AWS ALB), the client sends traffic to a physical proxy that chooses the destination server. In client-side load balancing (used by Spring Cloud Gateway via `lb://`), the client/gateway fetches the list of available servers from Eureka and selects the destination server directly.

**Intermediate Questions & Answers****Q9: Explain the end-to-end request lifecycle when a parent views their child's attendance.**

**Answer:** 1. Parent logs in; React stores JWT containing `studentId: 14`. 2. React `ParentDashboard.jsx` invokes `attendanceService.getAttendanceByStudent(14)`. 3. Axios interceptor attaches `Authorization: Bearer <token>` and sends `GET http://localhost:8080/api/attendance/student/14`. 4. API Gateway matches `/api/attendance/**`, queries Eureka for `ATTENDANCE-SERVICE`, and routes to `127.0.0.1:8084`. 5. `AttendanceController.getAttendanceByStudent()` delegates to `AttendanceService`. 6. `AttendanceRepository` queries `sms_attendance_db` table `attendances` where `student_id = 14`. 7. The database records are mapped to DTOs and returned as JSON through the Gateway to React.

**Q10: How does this project prevent a parent from submitting a leave request for a different student?**

**Answer:** In `AttendanceController.java` (`submitLeave`), the backend does not trust any `studentId` passed in the HTTP request body. Instead, it extracts the Authorization header, decrypts/validates the JWT using `JwtDecoder`, and extracts `studentId` directly from the cryptographically signed JWT claims. Since the client cannot forge the signature without the secret key, ownership is strictly enforced.

**Q11: What happens if the Eureka server crashes while the system is running?**

**Answer:** Because Spring Cloud Gateway and the microservices cache the Eureka registry locally, existing services can continue communicating through the Gateway for a short grace period. However, new instances cannot register, killed services will not be evicted, and a full system reboot will fail until Eureka is restored.

**Q12: Why did we configure `prefer-ip-address: true` and `ip-address: 127.0.0.1`?**

**Answer:** By default, Eureka attempts to resolve the hostname or primary network interface IP. On developer machines with active VPNs, Docker adapters, or dynamic Wi-Fi IPs, Eureka can register an unreachable IP. Explicitly configuring `127.0.0.1` ensures the API Gateway consistently connects to local loopback ports.

**Q13: How does the system handle CORS between React (:5173) and the Gateway (:8080)?**

**Answer:** In `api-gateway`, a `CorsWebFilter` bean is configured using `UrlBasedCorsConfigurationSource`. It intercepts browser `OPTIONS` preflight requests, approves origins matching `http://localhost:*` and `http://127.0.0.1:*`, allows HTTP methods (`GET`, `POST`, `PUT`, `DELETE`, etc.), and permits the `Authorization` header with `allowCredentials(true)`.

**Q14: What is the difference between synchronous REST and asynchronous event-driven messaging?**

**Answer:** Synchronous REST requires the caller to wait for the callee to finish and respond over an open HTTP socket. Asynchronous messaging (using Kafka or RabbitMQ) publishes an immutable event message to a queue or topic; consumers read and process the event independently without blocking the publisher.

**Q15: How are payments linked to fee invoices in `fee-service`?**

**Answer:** `Fee` and `Payment` entities reside in the same database (`sms_fee_db`). When `POST /api/fees/{feeId}/payments` is called, `FeeService.recordPayment()` creates a `Payment` entity with `feeId`, increments `paidAmount` on the parent `Fee`, decrements `pendingAmount`, and updates `status` to `PAID` or `PARTIAL` within a single `@Transactional` database operation.

**Q16: Why doesn't `academic-service` have a Spring Security dependency?**

**Answer:** `academic-service` relies on perimeter security managed by the API Gateway and stateless token validation where required. By omitting heavy Spring Security filters in internal services that only process sanitized Gateway traffic, application startup time and runtime overhead are minimized.

**Q17: How does the Teacher Gradebook interact with Student records across service boundaries?**

**Answer:** The teacher frontend fetches the student roster from `student-service` ( `GET /api/students` ) and the exams list from `academic-service` ( `GET /api/academic/exams` ). When the teacher enters grades, the frontend submits records containing `studentId`, `examId`, and `marksObtained` to `academic-service` ( `POST /api/academic/marks` ). The boundary is maintained through plain ID references.

**Q18: What production improvements would you introduce to this architecture?**

**Answer:** 1. **Resilience4j Circuit Breakers:** Protect the API Gateway from cascading failures when a microservice is unresponsive. 2. **Centralized Configuration:** Spring Cloud Config Server backed by Git for dynamic configuration updates. 3. **Distributed Tracing:** Micrometer Tracing with Zipkin to trace requests across microservices. 4. **Secret Management:** Move JWT secrets and database credentials to HashiCorp Vault or AWS Secrets Manager. 5. **Asynchronous Messaging:** Introduce Kafka or RabbitMQ for non-critical side effects (e.g. sending fee payment SMS receipts).

## Step 21 — 10-Day Structured Learning Sequence

To master this architecture step by step, follow this structured 10-day roadmap:

### Day 1: Microservices Fundamentals & Workspace Orientation

- **Study Goal:** Understand why this project is divided into microservices and how each service has its own database.
- **Files to Open:**
  - `school-management/pom.xml` (Review multi-module Maven layout).
  - `school-common/src/main/java/com/school/common/dto/ApiResponse.java`.
- **Hands-on Task:** Check PostgreSQL to see the 5 separate databases ( `sms_auth_db`, `sms_student_db`, `sms_academic_db`, `sms_attendance_db`, `sms_fee_db` ).

### Day 2: Service Registry & Eureka

- **Study Goal:** Learn how microservices register and discover each other.
- **Files to Open:**
  - `service-registry/src/main/resources/application.yml`.
  - `service-registry/src/main/java/com/school/registry/ServiceRegistryApplication.java`.
  - `student-service/src/main/resources/application.yml` (Eureka client block).
- **Hands-on Task:** Open `http://localhost:8761` in your browser. Inspect the Eureka dashboard and verify that all 6 microservices display status `UP (1)` - `127.0.0.1:<port>`.

### Day 3: Spring Cloud API Gateway & Routing

- **Study Goal:** Master reverse proxy routing, load balancer URIs ( `lb://` ), and CORS.
- **Files to Open:**
  - `api-gateway/src/main/resources/application.yml`.
  - `api-gateway/src/main/java/com/school/gateway/config/CorsConfig.java`.
- **Hands-on Task:** Execute in PowerShell: `powershell Invoke-RestMethod -Uri "http://localhost:8080/actuator/gateway/routes"` Inspect the registered dynamic route predicates.

### Day 4: Distributed JWT Authentication & Security

- **Study Goal:** Understand token creation, signing, claims, and stateless verification.
- **Files to Open:**
  - `auth-service/src/main/java/com/school/auth/security/JwtUtil.java`.
  - `auth-service/src/main/java/com/school/auth/controller/AuthController.java`.
  - `student-service/src/main/java/com/school/student/security/JwtDecoder.java`.
- **Hands-on Task:** Log in via Postman or PowerShell: `powershell $body = @{ email = "admin@school.com"; password = "admin" } | ConvertTo-Json`  
`Invoke-RestMethod -Uri "http://localhost:8080/api/auth/login" -Method Post -Body $body -ContentType "application/json"` Copy the token and paste it into [jwt.io](#) to inspect the payload claims ( `userId`, `role`, `email` ).

### Day 5: Axios Interceptors & React Gateway Integration

- **Study Goal:** Trace how the frontend dispatches authenticated calls to port `8080`.
- **Files to Open:**
  - `school-frontend/src/services/api.js`.
  - `school-frontend/src/services/studentService.js`.
- **Hands-on Task:** Open browser Developer Tools (F12) -> **Network Tab**. Click any menu in the application. Inspect the request URL ( `http://localhost:8080/...` ) and verify the `Authorization: Bearer` header.

## Day 6: Parent → Student Relationship & Ownership Security

- **Study Goal:** Understand cross-service entity references and token claim extraction.
- **Files to Open:**
  - `auth-service/src/main/java/com/school/auth/entity/User.java` (Notice `studentId`).
  - `student-service/src/main/java/com/school/student/controller/StudentController.java` ( `getStudentsByParent` ).
  - `attendance-service/src/main/java/com/school/attendance/controller/AttendanceController.java` ( `submitLeave` ).
- **Hands-on Task:** Trace how `submitLeave()` extracts `studentId` directly from the token without accepting it from the request body.

## Day 7: Teacher Command Center & Academic Assessment Flow

- **Study Goal:** Learn how the Teacher dashboard aggregates data from multiple microservices.
- **Files to Open:**
  - `school-frontend/src/pages/teacher/TeacherDashboard.jsx` .
  - `academic-service/src/main/java/com/school/academic/controller/AcademicController.java` .
  - `academic-service/src/main/java/com/school/academic/service/AcademicService.java` .
- **Hands-on Task:** Enter marks for a student in the Teacher portal. Verify the record in PostgreSQL: `sql SELECT * FROM marks ORDER BY id DESC LIMIT 5;`

## Day 8: Fee & Payment Transaction Processing

- **Study Goal:** Understand transactional ledger updates within a dedicated microservice.
- **Files to Open:**
  - `fee-service/src/main/java/com/school/fee/entity/Fee.java` .
  - `fee-service/src/main/java/com/school/fee/entity/Payment.java` .
  - `fee-service/src/main/java/com/school/fee/service/FeeService.java` .
  - `school-frontend/src/pages/admin/FeesPage.jsx` .
- **Hands-on Task:** Record a partial fee payment and observe how `paidAmount` , `pendingAmount` , and `status` update automatically.

## Day 9: Failure Analysis & Resilience

- **Study Goal:** Understand how microservices react when individual components fail.
- **Hands-on Task:**
  - Kill `attendance-service` .
  - Open the React frontend.
  - Notice that Student management and Fees still work! Only Attendance displays an error.
  - Restart `attendance-service` and watch it re-register with Eureka within 10 seconds.

## Day 10: Mock Technical Interview & Architecture Defense

- **Study Goal:** Practice explaining this project's architecture aloud using the questions in Step 20.
- **Review:**
  - Explain why you chose client-side aggregation over OpenFeign.
  - Explain why databases are private.
  - Explain how Eureka and Gateway collaborate using `lb://` .

# Step 22 — Verification & Audit Summary

## 22.1 Final Verification Checklist

- [x] **Zero Code Changes:** No existing Java classes, configuration files, POM files, or React components were modified or refactored.
- [x] **Accurate Technology Report:** Explicitly identified that OpenFeign, RestClient, Kafka, and direct backend-to-backend REST calls are **not currently used** in this project.
- [x] **Real Class & Endpoint Traces:** Used verified file paths, class names, method names, and routes from the workspace.
- [x] **Architecture Ground Truth:** Clearly articulated the Client-Side Aggregation + Stateless JWT Claim Propagation model.

## 22.2 Architectural Summary

The School Management System implements a **lightweight, practical, and maintainable microservices architecture** that balances enterprise decoupling with development simplicity. By delegating routing and CORS to Spring Cloud Gateway, discovery to Netflix Eureka, identity claims to signed JWTs, and view composition to the React frontend, the system avoids the overhead of distributed network calls while retaining all key benefits of independent service deployment and database isolation.